

CLINICAL DATA CLASSIFICATION SYSTEM: RULE-BASED PATIENT RECORD MANAGEMENT

Abdulmutin

Kastamonu University, Department of Computer Engineering, 2nd Year
Database Management Systems Course Project Report

Abstract- This report presents the Clinical Data Classification System, a web application developed with ASP.NET Core MVC, Entity Framework Core, and PostgreSQL. The project manages patient information, clinical records, and classification categories using a relational database. When a clinical record is created or updated, values such as pulse, blood pressure, blood sugar, cholesterol, body temperature, diagnosis, and notes are evaluated by a rule-based service. As a result, each record is assigned to one of the categories: Normal, Risky, Emergency, or Chronic Follow-Up Required. The application demonstrates database design, foreign key relationships, Code First migrations, seed data, CRUD operations, filtering, and dashboard statistics, making it suitable for a Database Management Systems course project.

Keywords- PostgreSQL, Entity Framework Core, ASP.NET Core MVC, Clinical Data, Classification, Database Management Systems.

I. INTRODUCTION

In healthcare-related applications, storing patient and clinical records in an organized way is essential for data-driven decision making. Patient identity data, examination records, measurement values, and risk categories are naturally related. Therefore, a relational database model is an appropriate design choice for clinical record management.

The Clinical Data Classification System was developed as an academic project for a Database Management Systems course. The project stores patient information and clinical measurements, while also classifying clinical records automatically. This makes the system more than a simple data entry application because records are assigned to meaningful medical risk categories.

II. PURPOSE AND SCOPE

The main purpose of the project is to design a relational database model for patients and clinical records and to perform complete CRUD operations on this model. Another objective is to evaluate clinical values using threshold-based rules and classify each record automatically.

The main goals of the project are:

- Store patient data with a unique patient number.
- Connect multiple clinical records to a single patient.
- Automatically classify clinical records.
- Manage classification categories in the database.
- Filter records by class, risk level, and date range.
- Display total and class-based statistics on the dashboard.

III. TECHNOLOGIES USED

The application is an ASP.NET Core MVC project targeting .NET 8. Entity Framework Core is used for database access. PostgreSQL is selected as the relational database system, and the Npgsql provider connects the application to PostgreSQL. The user interface is built with Razor Views and Bootstrap 5. Docker Compose is provided to run PostgreSQL locally.

Table I. Technologies Used

Technology	Purpose
ASP.NET Core MVC	Web architecture based on controllers, models, and views
Entity Framework Core	ORM, Code First migrations, and data access
PostgreSQL	Storage for patients, clinical records, and classes
Npgsql	Connection between .NET and PostgreSQL
Bootstrap 5	Responsive and clean user interface
Docker Compose	Simple local startup for PostgreSQL

IV. SYSTEM ARCHITECTURE

The project follows the MVC architecture. Controller classes handle HTTP requests and use `ApplicationDbContext` for database operations. The classification logic is not written directly inside the controller; it is separated through the `IClassificationService` interface and the `ClassificationService` class. This structure centralizes the business rule and improves code readability.

Table II. Folders and Responsibilities

Layer	Example Files	Responsibility
Models	Patient, ClinicalRecord, ClinicalClass	Database entities and validation rules
Data	ApplicationDbContext, SeedData	DbSet definitions, relationships, seed data
Services	ClassificationService	Rule-based automatic classification
Controllers	PatientsController, ClinicalRecordsController	CRUD and page flow
Views	Razor .cshtml files	User interface
Migrations	Migration files	Versioned database schema

V. DATABASE DESIGN

The database is built around three main tables: Patients, ClinicalRecords, and ClinicalClasses. The Patients table stores first name, last name, patient number, birth date, gender, phone, and address. The ClinicalRecords table stores complaint, diagnosis, blood pressure, pulse, body temperature, blood sugar, cholesterol, notes, and record date. The ClinicalClasses table stores categories such as Normal, Risky, Emergency, and Chronic Follow-Up Required.

One patient can have multiple clinical records. Therefore, a one-to-many relationship is defined between Patients and ClinicalRecords. Each clinical record is connected to one classification category through the ClinicalRecords.ClinicalClassId foreign key. DeleteBehavior.Restrict is used for clinical classes so that a class with related records cannot be deleted accidentally.

Table III. Database Tables

Table	Main Fields	Description
Patients	Id, FirstName, LastName, PatientNumber	Basic patient information
ClinicalRecords	PatientId, ClinicalClassId, Diagnosis, Pulse, BloodSugar	Clinical measurements and diagnosis records
ClinicalClasses	Name, RiskLevel, ColorCode	Classification categories and risk levels

VI. DATA INTEGRITY AND VALIDATION

A unique index is defined for PatientNumber, preventing duplicate patient numbers. The ClinicalClass.Name field is also protected with a unique index. Model classes use data annotation attributes such as Required, StringLength, Range, Phone, and DataType. These annotations improve data quality during model validation and form submission.

Range checks are used to prevent invalid negative clinical measurements. For example, pulse is validated between 0 and 250, body temperature between 0 and 45, and blood sugar and cholesterol between 0 and 1000.

VII. CLASSIFICATION ALGORITHM

The classification process is implemented in ClassificationService. When a clinical record is created or updated, the service returns the appropriate class name. The controller then finds the matching ClinicalClass record from the database and writes its id into ClinicalClassId.

The algorithm first checks pulse. If pulse is lower than 50 or higher than 120, the record is immediately classified as Emergency. Then, an abnormal value count is calculated using blood sugar, body temperature, blood pressure, and cholesterol.

$$\text{Abnormal Count} = \text{Blood Sugar} + \text{Fever} + \text{Blood Pressure} + \text{Cholesterol indicators}$$

Table IV. Classification Rules

Condition	Result
Pulse < 50 or > 120	Emergency
Blood sugar > 180	Abnormal count increases
Body temperature > 38	Abnormal count increases
Systolic blood pressure >= 140 or diastolic blood pressure >= 90	Abnormal count increases
Cholesterol > 240	Abnormal count increases
Abnormal count >= 2	Emergency
Diagnosis or notes include chronic/follow-up terms	Chronic Follow-Up Required
One abnormal value	Risky
No abnormal value	Normal

VIII. APPLICATION MODULES

The Dashboard module displays the total number of patients, total clinical records, record counts for Normal, Risky, and Emergency classes, and the latest clinical records. It also lists class statistics by category.

The Patient Management module includes listing, searching, creating, editing, detail viewing, and deleting patients. Search is performed by first name, last name, and patient number. The Clinical Record Management module includes CRUD operations, automatic classification, and filters by class, risk level, and date range. The Classification Categories module supports creating, editing, viewing, and deleting categories; however, categories with related clinical records cannot be deleted.

IX. SEED DATA AND MIGRATIONS

The project uses the Code First approach. Migration files version the database schema. Default classification categories, sample patients, and sample clinical records are defined with `HasData` in `ApplicationDbContext.OnModelCreating`. The `SeedData.cs` file also contains an example of runtime seeding.

The default classes are Normal, Risky, Emergency, and Chronic Follow-Up Required. Sample records include routine control, suspected hypertension, and chronic diabetes follow-up scenarios. This seed data makes the application testable immediately after database creation.

X. SAMPLE SCENARIO

When a user creates a clinical record for Mehmet Demir with headache and fatigue complaints, the system evaluates blood pressure and other measurements. Since systolic blood pressure is 152 and diastolic blood pressure

is 96, the blood pressure indicator is considered abnormal. If more than one abnormal value exists, the record is classified as Emergency; if only one abnormal value exists, it is classified as Risky. If the diagnosis or notes include chronic or follow-up terms, the record is classified as Chronic Follow-Up Required.

XI. CONCLUSION

The Clinical Data Classification System is a practical Database Management Systems project that demonstrates relational database design, data integrity, migrations, seed data, and CRUD operations. Developed with ASP.NET Core MVC and Entity Framework Core, the system manages patient and clinical records on PostgreSQL. The rule-based classification service gives additional meaning to stored clinical data by assigning records to risk categories. The project can be extended with user authorization, advanced reporting, charts, and more detailed clinical decision rules.

REFERENCES

- [1] Microsoft, "ASP.NET Core MVC documentation," Microsoft Learn.
- [2] Microsoft, "Entity Framework Core documentation," Microsoft Learn.
- [3] PostgreSQL Global Development Group, "PostgreSQL Documentation."
- [4] Clinical Data Classification System project source code and README file.